

- 1) Configure CORS for React dev origin and verify they can call **GET /products** from the frontend using Axios.

```
app = FastAPI(title="ShopHub Product API", version="1.1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["*"],
)
```

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/products?page=1&size=10' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/products?page=1&size=10
```

Server response

Code Details

200

Response body

```
{
  "total": 5,
  "page": 1,
  "size": 10,
  "items": [
    {
      "id": 1,
      "name": "Fjallraven Everyday Backpack",
      "price": 109.95,
      "category": "Bags",
      "description": "Durable everyday backpack with a padded laptop sleeve and roomy storage for school, work, or travel.",
      "imageUrl": "https://fakestoreapi.com/img/81fPKd-2AYL_AC_SL1500_.jpg"
    },
    {
      "id": 2,
      "name": "Men's Casual Premium Slim Fit T-Shirt",
      "price": 22.9,
      "category": "Men's Clothing",
      "description": "Soft slim-fit cotton blend shirt with contrast raglan sleeves and a comfortable lightweight feel.",
      "imageUrl": "https://fakestoreapi.com/img/71-3HjONUL_AC_SY879._SX._UX._SY._UY_.jpg"
    },
    {
      "id": 3,
      "name": "Men's Cotton Jacket",
      "price": 55.99,
      "category": "Men's Clothing",
      "description": "Lightweight outerwear jacket suitable for spring, autumn, hiking, camping, and casual daily wear.",
      "imageUrl": "https://fakestoreapi.com/img/7111-ujtIUU_AC_UX679_.jpg"
    }
  ]
}
```

Response headers

```
content-length: 1401
content-type: application/json
date: Thu, 25 Jun 2026 14:45:16 GMT
server: uvicorn
```

2. Implement input/output schemas ([ProductCreate](#), [ProductUpdate](#), [ProductPublic](#)) and use [response_model](#) to hide sensitive fields (e.g. [costPrice](#)).

```
class Product(BaseModel):
    id: int
    name: str = Field(..., min_length=3, max_length=100)
    price: float = Field(..., gt=0)
    category: str = Field(..., min_length=3, max_length=50)
    description: str = Field(..., min_length=5)
    imagePath: str = Field(..., min_length=1)
    costPrice: Optional[float] = None

    @field_validator("name", "category", "description", "imagePath", mode="before")
    @classmethod
    def trim_required_text(cls, value):
        if isinstance(value, str):
            return value.strip()
        return value

class ProductCreate(BaseModel):
    name: str = Field(..., min_length=3, max_length=100)
    price: float = Field(..., gt=0)
    category: str = Field(..., min_length=3, max_length=50)
    description: str = Field(..., min_length=5)
    imageUrl: Optional[str] = None

    @field_validator("name", "category", "description", mode="before")
    @classmethod
    def trim_required_text(cls, value):
        if isinstance(value, str):
            return value.strip()
        return value

    @field_validator("imageUrl", mode="before")
    @classmethod
    def trim_optional_url(cls, value):
        if value is None:
            return None
        if isinstance(value, str):
            value = value.strip()
            return value or None
        return value
```

```
class ProductUpdate(BaseModel):
    name: Optional[str] = Field(None, min_length=3, max_length=100)
    price: Optional[float] = Field(None, gt=0)
    category: Optional[str] = Field(None, min_length=3, max_length=50)
    description: Optional[str] = Field(None, min_length=5)
    imageUrl: Optional[str] = None

    @field_validator("name", "category", "description", mode="before")
    @classmethod
    def trim_optional_text(cls, value):
        if value is None:
            return None
        if isinstance(value, str):
            return value.strip()
        return value

    @field_validator("imageUrl", mode="before")
    @classmethod
    def trim_optional_url(cls, value):
        if value is None:
            return None
        if isinstance(value, str):
            value = value.strip()
            return value or None
        return value
```

```

class ProductPublic(BaseModel):
    id: int
    name: str
    price: float
    category: str
    description: str
    imageUrl: str

class ProductListResponse(BaseModel):
    total: int
    page: int
    size: int
    items: list[ProductPublic]

SEED_PRODUCTS = [
    {
        "id": 1,
        "name": "Fjallraven Everyday Backpack",
        "price": 109.95,
        "category": "Bags",
        "description": "Durable everyday backpack with a padded laptop sleeve and roomy storage for school, work, or travel.",
        "imagePath": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_.jpg",
        "costPrice": 65.0,
    },
    {
        "id": 2,
        "name": "Mens Casual Premium Slim Fit T-Shirt",
        "price": 22.30,
        "category": "Men's Clothing",
        "description": "Soft slim-fit cotton blend shirt with contrast raglan sleeves and a comfortable lightweight feel.",
    },

```

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/products?page=1&size=10' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/products?page=1&size=10

Server response

Code Details

200

Response body

```

{
  "total": 5,
  "page": 1,
  "size": 10,
  "items": [
    {
      "id": 1,
      "name": "Fjallraven Everyday Backpack",
      "price": 109.95,
      "category": "Bags",
      "description": "Durable everyday backpack with a padded laptop sleeve and roomy storage for school, work, or travel.",
      "imageUrl": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_.jpg"
    },
    {
      "id": 2,
      "name": "Mens Casual Premium Slim Fit T-Shirt",
      "price": 22.3,
      "category": "Men's Clothing",
      "description": "Soft slim-fit cotton blend shirt with contrast raglan sleeves and a comfortable lightweight feel.",
      "imageUrl": "https://fakestoreapi.com/img/71-3HjGNDUL._AC_SY879._SX._UX._SY._UY_.jpg"
    },
    {
      "id": 3,
      "name": "Mens Cotton Jacket",
      "price": 55.99,
      "category": "Men's Clothing",
      "description": "Lightweight outerwear jacket suitable for spring, autumn, hiking, camping, and casual daily wear.",
      "imageUrl": "https://fakestoreapi.com/img/7111-ujCUIUL._AC_UX679_.jpg"
    }
  ]
}
```

Response headers

```

content-length: 1401
content-type: application/json
date: Thu, 25 Jun 2026 19:19:58 GMT
server: uvicorn
```

3. Implement:

- POST /products with optional image upload.
- PUT /products/{id} to update product and image.

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/products' \
  -H 'accept: application/json' \
  -H 'Content-type: multipart/form-data' \
  -F 'name=Uploaded Image Product' \
  -F 'price=150' \
  -F 'category=Upload Test' \
  -F 'description=This product was created with an uploaded image file.' \
  -F 'imageUrl=leave empty' \
  -F 'image_file=@image.webp;type=image/webp'
```

Request URL

http://127.0.0.1:8000/products

Server response

Code	Details
201	Response body <pre>{ "id": 7, "name": "Uploaded Image Product", "price": 150, "category": "Upload Test", "description": "This product was created with an uploaded image file.", "imageUrl": "/images/552c7c72c4d94c53892ee176b3377465_image.webp" }</pre> Response headers <pre>access-control-allow-credentials: true content-length: 214 content-type: application/json date: Thu, 25 Jun 2026 19:53:35 GMT server: uvicorn</pre>

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/products' \
  -H 'accept: application/json' \
  -H 'Content-type: multipart/form-data' \
  -F 'name=Fjallraven Everyday Backpack' \
  -F 'price=109.95' \
  -F 'category=Bags' \
  -F 'description=Durable everyday backpack with a padded laptop sleeve and roomy storage for school, work, or travel.' \
  -F 'imageUrl=https://fakestoreapi.com/img/81fPKd-2AYL._AC_S1500_.jpg' \
  -F 'image_file='
```

Request URL

http://127.0.0.1:8000/products

Server response

Code	Details
201	Response body <pre>{ "id": 6, "name": "Fjallraven Everyday Backpack", "price": 109.95, "category": "Bags", "description": "Durable everyday backpack with a padded laptop sleeve and roomy storage for school, work, or travel.", "imageUrl": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_S1500_.jpg" }</pre> Response headers <pre>access-control-allow-credentials: true content-length: 266 content-type: application/json date: Thu, 25 Jun 2026 19:48:20 GMT server: uvicorn</pre>

Put product :

The screenshot shows a REST client interface with the following details:

- Curl:**

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/products/6' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'name=This is test 1 product 6' \
  -F 'price=111' \
  -F 'category=Bags' \
  -F 'description=test 1' \
  -F 'imageUrl=This product was created during Session 06 API testing' \
  -F 'image_file'
```
- Request URL:** `http://127.0.0.1:8000/products/6`
- Server response:** 200
- Response body:**

```
{
  "id": 6,
  "name": "This is test 1 product 6",
  "price": 111,
  "category": "Bags",
  "description": "test 1",
  "imageUrl": "/images/This product was created during Session 06 API testing"
}
```
- Response headers:**

```
access-control-allow-credentials: true
content-length: 173
content-type: application/json
date: Thu, 25 Jun 2026 19:56:51 GMT
server: uvicorn
```

4. Implement `GET /products` with:

a) `category`, `min_price`, `max_price`, `page`, `size` as query parameters.

The screenshot shows a REST client interface with the following details:

- Response body:**

```
{
  "total": 7,
  "page": 1,
  "size": 2,
  "items": [
    {
      "id": 1,
      "name": "Fjallraven Everyday Backpack",
      "price": 109.95,
      "category": "Bags",
      "description": "Durable everyday backpack with a padded laptop sleeve and roomy storage for school, work, or travel.",
      "imageUrl": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_S11500_.jpg"
    },
    {
      "id": 2,
      "name": "Mens Casual Premium Slim Fit T-Shirt",
      "price": 22.3,
      "category": "Men's Clothing",
      "description": "Soft slim-fit cotton blend shirt with contrast raglan sleeves and a comfortable lightweight feel.",
      "imageUrl": "https://fakestoreapi.com/img/71-3HjGNDUL._AC_SY879._SX._UX._SY._UY_.jpg"
    }
  ]
}
```
- Response headers:**

```
content-length: 601
content-type: application/json
date: Thu, 25 Jun 2026 20:03:36 GMT
server: uvicorn
```

`GET /products` supports pagination using `page` and `size` query parameters. The response shows page 1, size 2, and only returns the requested number of products.

200

Response body

```
{
  "total": 2,
  "page": 1,
  "size": 10,
  "items": [
    {
      "id": 1,
      "name": "Fjallraven Everyday Backpack",
      "price": 109.95,
      "category": "Bags",
      "description": "Durable everyday backpack with a padded laptop sleeve and roomy storage for school, work, or travel.",
      "imageUrl": "https://fakestoreapi.com/img/81fPKd-2AYL_AC_SL1500_.jpg"
    },
    {
      "id": 6,
      "name": "This is test 1 product 6",
      "price": 111,
      "category": "Bags",
      "description": "test 1",
      "imageUrl": "/images/This product was created during Session 06 API testing"
    }
  ]
}
```

Response headers

```
content-length: 481
content-type: application/json
date: Thu, 25 Jun 2026 20:04:57 GMT
server: uvicorn
```

Download

GET /products supports category filtering. When category is set to Bags, the response only returns products from the Bags category.

200

Response body

```
{
  "total": 2,
  "page": 1,
  "size": 10,
  "items": [
    {
      "id": 2,
      "name": "Mens Casual Premium Slim Fit T-Shirt",
      "price": 22.3,
      "category": "Men's Clothing",
      "description": "Soft slim-fit cotton blend shirt with contrast raglan sleeves and a comfortable lightweight feel.",
      "imageUrl": "https://fakestoreapi.com/img/71-3HjGDUL_AC_SY879_SX_UX_UY_.jpg"
    },
    {
      "id": 3,
      "name": "Mens Cotton Jacket",
      "price": 55.99,
      "category": "Men's Clothing",
      "description": "Lightweight outerwear jacket suitable for spring, autumn, hiking, camping, and casual daily wear.",
      "imageUrl": "https://fakestoreapi.com/img/7111-ujtIUU_AC_UX679_.jpg"
    }
  ]
}
```

Response headers

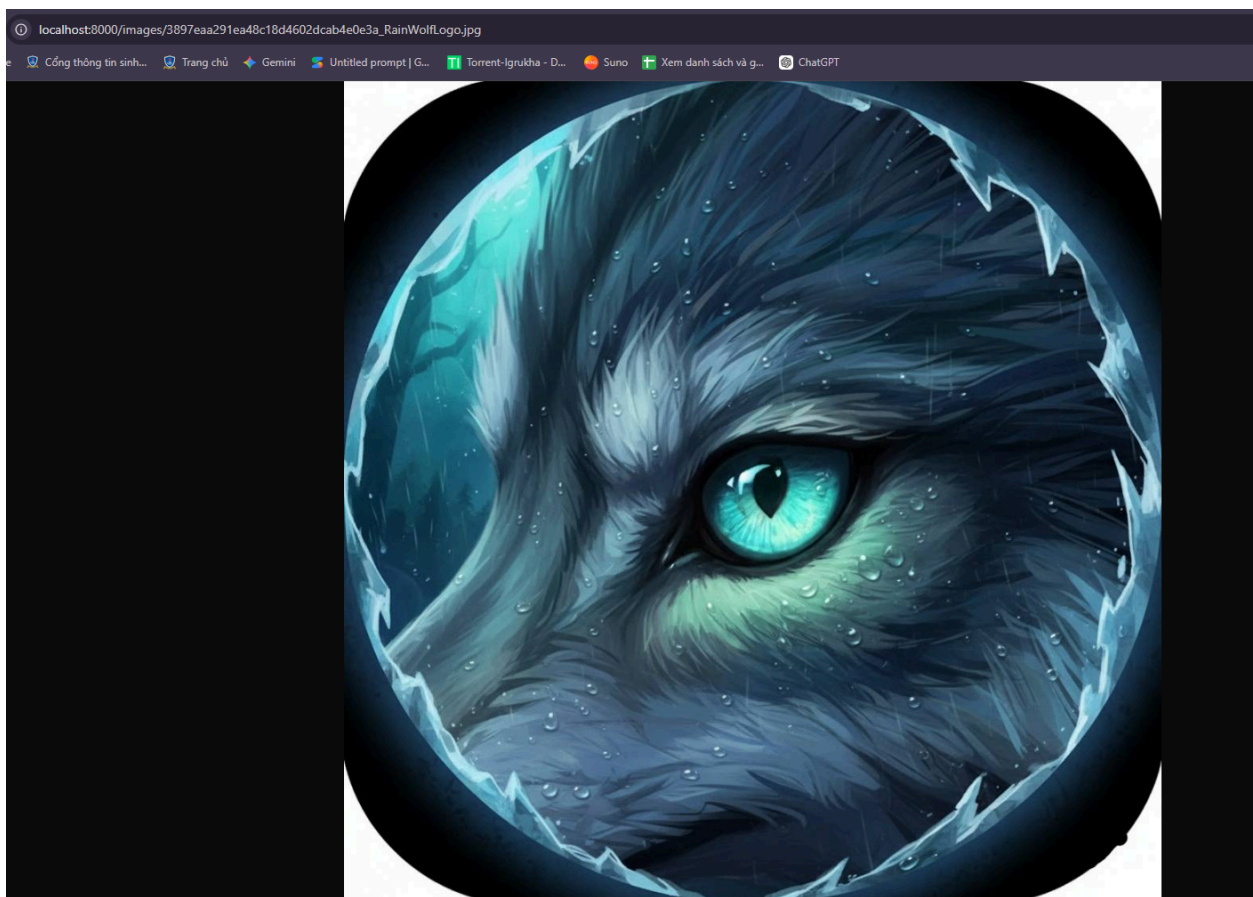
```
content-length: 597
content-type: application/json
date: Thu, 25 Jun 2026 20:06:26 GMT
server: uvicorn
```

Download

GET /products supports min_price and max_price query parameters. The response only includes products within the selected price range.

5 . Implement auxiliary `GET /images/{filename}` and confirm images can be displayed in the browser.

```
201
Response body
{
  "id": 0,
  "name": "Image Test Product",
  "price": 1.22,
  "category": "Image Test",
  "description": "This product is used to test image upload and image display.",
  "imageUrl": "/images/3897eaa291ea48c18d4602dcab4e0e3a_RainWolflLogo.jpg"
}
Response headers
access-control-allow-credentials: true
content-length: 222
content-type: application/json
date: Thu, 25 Jun 2026 20:11:39 GMT
server: uvicorn
```



The auxiliary `GET /images/{filename}` endpoint successfully displays the uploaded product image in the browser.

6. Add validation errors for:

- Missing image (no `imageUrl`, no `image_file`).
- Invalid price or category.

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/products' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'name=Missing Image Product' \
  -F 'price=50' \
  -F 'category=Test Category' \
  -F 'description=This product is missing an image.' \
  -F 'imageUrl=' \
  -F 'image_file='
```

Request URL

`http://127.0.0.1:8000/products`

Server response

Code	Details
422	Error: Unprocessable Content

Response body

```
{
  "detail": "Product image is required. Provide imageUrl or image_file."
}
```

Response headers

```
access-control-allow-credentials: true
content-length: 71
content-type: application/json
date: Thu, 25 Jun 2026 20:15:45 GMT
server: uvicorn
```

The API returns a validation error when creating a product without `imageUrl` or `image_file`.

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/products' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'name=Invalid Price Product' \
  -F 'price=-100' \
  -F 'category=Test Category' \
  -F 'description=This product has an invalid negative price.' \
  -F 'imageUrl=https://via.placeholder.com/300' \
  -F 'image_file='
```

Request URL

`http://127.0.0.1:8000/products`

Server response

Code	Details
422	Error: Unprocessable Content

Response body

```
{
  "detail": [
    {
      "type": "greater_than",
      "loc": [
        "price"
      ],
      "msg": "Input should be greater than 0",
      "input": -100,
      "ctx": {
        "gt": 0
      }
    }
  ],
  "url": "https://errors.pydantic.dev/2.13/v/greater_than"
}
```

Response headers

```
access-control-allow-credentials: true
content-length: 179
content-type: application/json
date: Thu, 25 Jun 2026 20:45:57 GMT
server: uvicorn
```

The API validates product price and rejects invalid values such as negative prices.

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/products' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'name=Invalid Category Product' \
  -F 'price=50' \
  -F 'category=A' \
  -F 'description=This product has an invalid short category.' \
  -F 'imageurl=https://via.placeholder.com/300' \
  -F 'image_file='
```

Request URL

http://127.0.0.1:8000/products

Server response

Code	Details
422	Error: Unprocessable Content

Response body

```
{
  "detail": [
    {
      "type": "string_too_short",
      "loc": [
        "category"
      ],
      "msg": "String should have at least 3 characters",
      "input": "A",
      "ctx": {
        "min_length": 3
      },
      "url": "https://errors.pydantic.dev/2.13/v/string_too_short"
    }
  ]
}
```

Response headers

```
access-control-allow-credentials: true
content-length: 203
content-type: application/json
date: Thu, 25 Jun 2026 20:47:13 GMT
server: uvicorn
```

The API validates product category and rejects invalid category values.

7. Add a query parameter **search** to filter by product name or description text.

GET /products Get Products

Parameters Cancel

Name	Description
category string (string null) (query)	category
min_price number (number null) (query)	min_price minimum: 0
max_price number (number null) (query)	max_price minimum: 0
search string (string null) (query)	backpack
page integer (query)	1 minimum: 1
size integer (query)	10 maximum: 100 minimum: 1

Execute Clear

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/products?search=backpack&page=1&size=10' \
  -H 'accept: application/json'
```

Request URL

<http://127.0.0.1:8000/products?search=backpack&page=1&size=10>

Server response

Code Details

200

Response body

```
{
  "total": 1,
  "page": 1,
  "size": 10,
  "items": [
    {
      "id": 1,
      "name": "Fjallraven Everyday Backpack",
      "price": 109.95,
      "category": "Bags",
      "description": "Durable everyday backpack with a padded laptop sleeve and roomy storage for school, work, or travel.",
      "imageUrl": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_S1500_.jpg"
    }
  ]
}
```

Response headers

```
content-length: 307
content-type: application/json
date: Thu, 25 Jun 2026 21:11:48 GMT
server: uvicorn
```

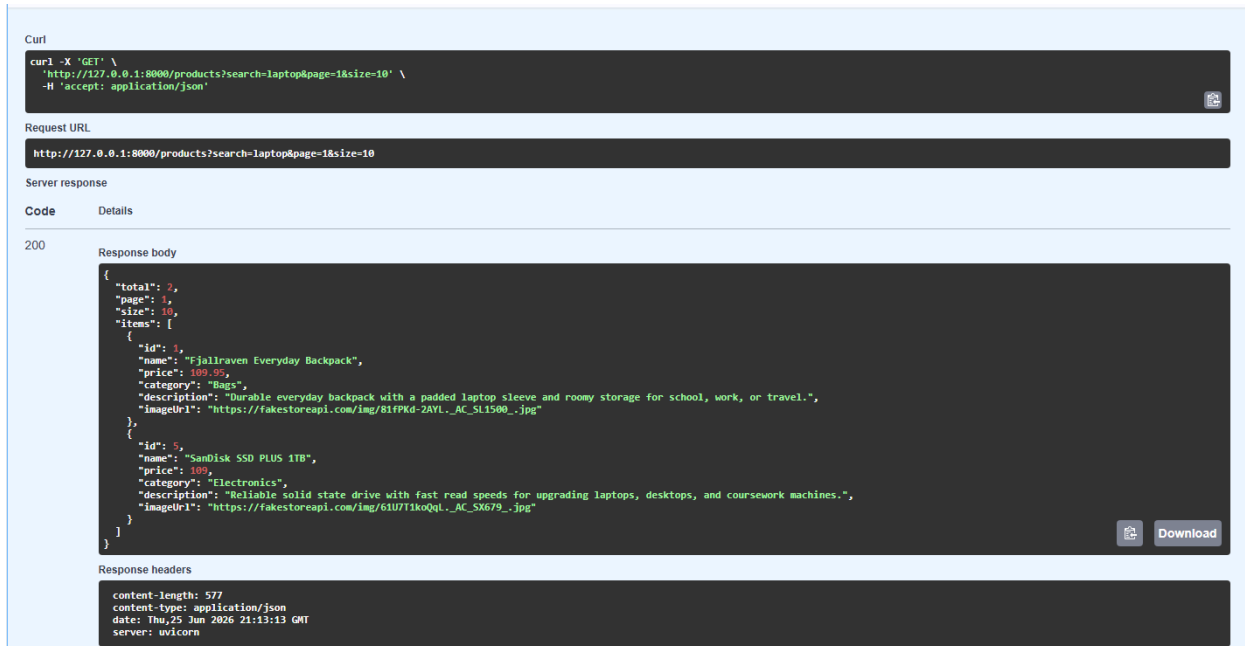
GET /products supports the search query parameter. Searching for “backpack” returns products whose name contains the search text.

GET /products Get Products

Parameters Cancel

Name	Description
category	category
string (string null) (query)	
min_price	min_price
number (number null) (query)	minimum: 0
max_price	max_price
number (number null) (query)	minimum: 0
search	laptop
string (string null) (query)	
page	1
integer (query)	minimum: 1
size	10
integer (query)	maximum: 100 minimum: 1

Execute Clear



The screenshot shows a web browser interface with a dark theme. At the top, a 'Curl' section displays a cURL command: `curl -X 'GET' \ 'http://127.0.0.1:8000/products?search=laptop&page=1&size=10' \ -H 'accept: application/json'`. Below this, the 'Request URL' is `http://127.0.0.1:8000/products?search=laptop&page=1&size=10`. The 'Server response' section shows a status code of 200. The 'Response body' is a JSON object with the following structure:

```
{  "total": 2,  "page": 1,  "size": 10,  "items": [    {      "id": 1,      "name": "Fjallraven Everyday Backpack",      "price": 409.95,      "category": "Bags",      "description": "Durable everyday backpack with a padded laptop sleeve and roomy storage for school, work, or travel.",      "imageUrl": "https://fakestoreapi.com/img/81fPKd-2AYL_AC_SL1500_.jpg"    },    {      "id": 5,      "name": "SanDisk SSD PLUS 1TB",      "price": 109,      "category": "Electronics",      "description": "Reliable solid state drive with fast read speeds for upgrading laptops, desktops, and coursework machines.",      "imageUrl": "https://fakestoreapi.com/img/61U7T1koQQL_AC_SL679_.jpg"    }  ]}
```

. The 'Response headers' section shows: `content-length: 577`, `content-type: application/json`, `date: Thu, 25 Jun 2026 21:13:13 GMT`, and `server: uvicorn`. A 'Download' button is visible next to the response body.

The search parameter also filters by product description, not only product name.

8. Write a short summary explaining:

- Why separating input and output schemas is useful.
- How CORS, file upload, and pagination make the API more “real-world”.

Separating input and output schemas is useful because the API can clearly control what data users are allowed to send and what data they are allowed to receive. For example, `ProductCreate` and `ProductUpdate` are used for creating and updating products, while `ProductPublic` is used for responses. This keeps the API safer because internal fields such as `costPrice` can be stored in the backend but hidden from public responses.

CORS, file upload, and pagination make the API more real-world. CORS allows the React frontend to safely call the FastAPI backend from a different port during development. File upload allows product images to be stored on the server and displayed through image URLs, which is common in e-commerce websites. Pagination helps the

API return products in smaller groups instead of sending all data at once, making the product list easier to load and manage.